

Sistemas de Entrada e Saída (E/S)

FICHAS DE ESTUDO PARA REVISÃO DINÂMICA • PROF. FILIPO NOVO MÓR

01 Capa: Sistemas de E/S

Visão Geral: Introdução ao gerenciamento e estrutura de Entrada e Saída na disciplina de Sistemas Operacionais.

- **Foco Central:** Como o sistema operacional interage com o ambiente externo e periféricos.
- **Importância:** Viabilizar que o software controle o hardware de forma segura, abstrata e eficiente.

Contexto Prático: Sem os sistemas de E/S, o computador não seria capaz de ler dados de usuários ou exibir respostas, agindo apenas como um processador isolado.

Prof. Filipo Novo Mór • Sistemas Operacionais

02 Objetivos da Apresentação

Metas de Aprendizado:

- Explorar a estrutura interna do subsistema de E/S residente no Kernel.
- Discutir os princípios do hardware de E/S e sua inerente complexidade e variedade.
- Analisar aspectos cruciais de desempenho tanto em hardware quanto em software.
- Compreender a interface abstrata entre as aplicações e os dispositivos reais.

Prof. Filipo Novo Mór • Sistemas Operacionais

03 Hardware de E/S: Conceitos Básicos

Pilares de Conexão Física e Lógica:

- **Porta:** Ponto físico ou lógico de conexão para o dispositivo se comunicar.
- **Barramento (Bus):** Conjunto de fios físicos e um protocolo de comunicação rígido que transporta as mensagens.
- **Controlador (Adaptador):** Placa de circuitos eletrônicos responsável por operar e traduzir os sinais da porta ou barramento.

Exemplo Prático: No caso de um pendrive, o conector metálico é a **porta**, as trilhas internas da placa-mãe formam o **barramento** USB, e o chip interno do pendrive é o **controlador**.

Prof. Filipo Novo Mór • Sistemas Operacionais

04 Estrutura Típica de Barramento PC

Hierarquia de Velocidades:

- **Componentes de Alta Velocidade:** Processador e Memória RAM conectam-se no topo da hierarquia de barramentos do sistema (historicamente via Northbridge / MCH).
- **Barramento de Expansão (PCI / PCIe):** Interconecta periféricos mais lentos de forma organizada para não congestionar a CPU.
- **SCSI:** Interface dedicada a alto desempenho em discos de armazenamento.

Prof. Filipo Novo Mór • Sistemas Operacionais

05 Registradores de Porta de E/S

Comunicação de Baixo Nível (Geralmente 4 Registradores):

- **Data-in:** Lido pelo host (CPU) para obter os dados de entrada gerados pelo dispositivo.
- **Data-out:** Escrito pelo host para enviar dados de saída para o dispositivo.
- **Status:** Contém bits que indicam o estado atual (ex: se está ocupado, pronto ou em erro).
- **Control:** Escrito pelo host para iniciar comandos ou alterar modos de operação.

Prof. Filipo Novo Mór • Sistemas Operacionais

06 Polling (Consulta Ativa)

Mecanismo de Handshaking:

- O host lê repetidamente o bit de *status ocupado* (*busy bit*) até que ele se torne zero.
- **Eficiência:** É uma abordagem aceitável se o dispositivo for extremamente rápido.
- **Problema:** Se a espera for longa, a CPU desperdiça ciclos preciosos de processamento em um loop travado (chamado de *busy-wait*).

Prof. Filipo Novo Mór • Sistemas Operacionais

07 Mecanismo de Interrupções

Alternativa Eficiente ao Polling:

- A CPU possui uma linha física de solicitação de interrupção monitorada após cada instrução.
- O controlador do dispositivo sinaliza nessa linha quando conclui uma tarefa.
- A CPU faz o salvamento de estado (*state save*) do processo atual e salta imediatamente para o manipulador de interrupções (*interrupt handler*).

Exemplo Prático: Em vez de ficar olhando a tela do celular a cada minuto (Polling), você ativa o som de notificação (Interrupção) e vai fazer outra atividade.

Prof. Filipo Novo Mór • Sistemas Operacionais

08 Ciclo de E/S Dirigido por Interrupção

As Etapas do Fluxo Assíncrono:

1. O driver do dispositivo emite um comando de E/S.
2. O controlador executa e, ao finalizar, emite o sinal físico de interrupção.
3. A CPU captura o sinal, interrompe o fluxo de execução atual e trata o dado recebido.
4. A CPU retoma a execução normal do processo interrompido.

Prof. Filipo Novo Mór • Sistemas Operacionais

09 Interrupções: Recursos Avançados

Gerenciamento Complexo de Sinais:

- **Vetor de Interrupção:** Tabela de ponteiros de memória que despacha a CPU direto para a rotina correta sem precisar consultar o hardware.
- **Mascaramento:** Capacidade da CPU de adiar temporariamente interrupções não críticas durante tarefas de alta prioridade.
- **Níveis de Prioridade:** Permite que dispositivos urgentes (ex: falha de energia) interrompam tarefas menos urgentes (ex: teclado).

Prof. Filipo Novo Mór • Sistemas Operacionais

10 Tabela de Vetores (Intel Pentium)

Exemplo Real de Arquitetura de Hardware:

- Mapeamento fixo de vetores de 0 a 255.
- **Vetores 0 a 31:** Reservados para exceções internas da própria CPU (ex: divisão por zero, estouro de memória, ou *Page Fault*/Falha de Página).
- **Vetores 32 a 255:** Destinados a interrupções externas geradas por periféricos e dispositivos de E/S.

Prof. Filipo Novo Mór • Sistemas Operacionais

11 DMA: Direct Memory Access

Acesso Direto à Memória:

- Evita o uso de E/S programada (onde a CPU move byte por byte).
- O controlador de DMA transfere blocos inteiros de dados diretamente entre o periférico de alta velocidade e a memória RAM.
- A CPU fica totalmente liberada para computar outras tarefas durante a transferência massiva.

Exemplo Prático: Ao ler um filme em resolução 4K do seu SSD, o DMA joga os gigabytes de dados direto na memória sem que o processador precise carregar cada frame individualmente.

Prof. Filipo Novo Mór • Sistemas Operacionais

12 Fluxo de Transferência DMA

Coreografia do Fluxo DMA:

- 1. A CPU configura o controlador DMA (especificando endereços de origem, destino e tamanho).
- 2. O DMA comanda o controlador do dispositivo a ler/escrever na memória.
- 3. Os dados trafegam diretamente pelo barramento.
- 4. Ao fim do bloco, o controlador DMA gera **uma única interrupção** para avisar a CPU do término.

Prof. Filipo Novo Mór • Sistemas Operacionais

13 Interface de E/S de Aplicação

Abstração de Software:

- O Kernel do Sistema Operacional encapsula a imensa diversidade de hardwares em **classes genéricas** de dispositivos.
- A camada de **Device Driver (Driver de Dispositivo)** esconde os detalhes específicos e comandos proprietários dos controladores de hardware.
- Garante que o SO e as aplicações criadas sejam agnósticas e independentes do fabricante do hardware.

Prof. Filipo Novo Mór • Sistemas Operacionais

14 Estrutura de E/S do Kernel

Arquitetura em Camadas:

- **Nível de Usuário:** Aplicações fazem chamadas de sistema padronizadas.
- **Subsistema de E/S do Kernel:** Gerencia segurança, buffers, cache e escalonamento de requisições.
- **Drivers:** Traduzem a lógica do SO para comandos do dispositivo.
- **Hardware:** Controladores físicos executam a operação elétrica.

Prof. Filipo Novo Mór • Sistemas Operacionais

15 Características dos Dispositivos

Dimensões de Variação de Hardware:

- **Unidade de Transferência:** Fluxo de caracteres isolados (teclado) vs Blocos estruturados (discos).
- **Modo de Acesso:** Sequencial (fitas magnéticas de backup) vs Aleatório (SSDs).
- **Tempo de Transferência:** Síncrono (previsível/coordenado) vs Assíncrono (eventos imprevisíveis).
- **Compartilhamento:** Compartilhável por vários processos simultâneos vs Dedicado a um único processo por vez.

Prof. Filipo Novo Mór • Sistemas Operacionais

16 Dispositivos de Bloco e Caractere

Divisão de Categorias da Interface:

- **Dispositivos de Bloco:** Operam via comandos estruturados como `read`, `write` e `seek` (posicionamento).
- **Dispositivos de Caractere:** Operam via fluxos lineares de bytes usando `get` e `put`.
- **Chamada `ioctl()`:** Sistema de "escape" genérico usado para enviar comandos proprietários e específicos que não entram no padrão de leitura/escrita de dados.

Exemplo Prático: O comando para ejetar uma gaveta de drive de mídia física é enviado através de uma chamada de sistema `ioctl()`.

Prof. Filipo Novo Mór • Sistemas Operacionais

17 Dispositivos de Rede

Interface Diferenciada (Sockets):

- Diferem drasticamente de discos e terminais, exigindo uma interface própria baseada em conexões lógicas e protocolos.
- **Função `select()`:** Permite que uma aplicação monitore múltiplos sockets de rede ao mesmo tempo, alertando quando qualquer um deles contiver dados.

Exemplo Prático: Servidores web de alta escala usam a lógica do `select()` ou `epoll()` para gerenciar milhares de conexões de usuários sem travar o processamento principal.

Prof. Filipo Novo Mór • Sistemas Operacionais

18 Relógios e Timers

O Tempo como Recurso de E/S:

- **Hora Atual:** Fornece a estampa de tempo real para o sistema de arquivos e aplicações.
- **Tempo Decorrido:** Usado pelo Kernel para monitorar prazos e tempos de resposta.
- **Timer Programável:** Gera interrupções periódicas elétricas cruciais para o escalonador preemptivo realizar as trocas de contexto (*time-slice*).

Prof. Filipo Novo Mór • Sistemas Operacionais

19 E/S Bloqueante (Blocking I/O)

Comportamento Padrão de Sistemas:

- A execução do processo do usuário é totalmente suspensa pelo Kernel.
- O processo sai da fila de prontos e vai para a fila de espera (bloqueados) associada ao dispositivo.
- **Vantagem:** Simplicidade linear para codificação e entendimento do fluxo do software.

Exemplo Prático: Chamar uma função de leitura de teclado tradicional suspende o programa; nenhuma linha de código seguinte roda até que o usuário aperte 'Enter'.

Prof. Filipo Novo Mór • Sistemas Operacionais

20 E/S Não Bloqueante

Responsividade Imediata:

- A chamada de sistema feita pela aplicação retorna imediatamente, sem suspender a thread.
- Retorna de forma instantânea com o número de bytes que estavam prontos no buffer no momento exato da chamada (ou código de erro indicando que não há nada pronto).
- Geralmente coordenada e implementada através de arquiteturas multithreading.

Prof. Filipo Novo Mór • Sistemas Operacionais

21 E/S Assíncrona (Asynchronous I/O)

Operação Paralela Real:

- O processo faz a solicitação e continua sua execução normal sem interrupções imediatas.
- A transferência física de E/S ocorre de forma totalmente independente em segundo plano.
- O subsistema de E/S do Kernel dispara um sinal ou evento para avisar o processo original apenas quando a operação completa for totalmente concluída na memória.

Prof. Filipo Novo Mór • Sistemas Operacionais

22 Subsistema de E/S do Kernel

Principais Serviços Internos de Otimização:

- **Escalonamento de E/S:** Reordena a fila de acessos aos dispositivos físicos para maximizar a vazão e eficiência do hardware.
- **Bufferização:** Guarda os dados temporariamente em memória RAM durante as transferências.
- **Caching:** Mantém cópias em memória super rápida para acelerar o acesso a dados lidos frequentemente.

Prof. Filipo Novo Mór • Sistemas Operacionais

23 Por que Bufferizar?

Três Motivos Técnicos Cruciais:

- **Mismatch de velocidade:** Adaptar a transferência entre dispositivos rápidos e lentos (ex: internet vs gravação em disco).
- **Mismatch de tamanho:** Lidar com fluxos que coletam tamanhos de dados diferentes dos blocos exigidos pelo destino.
- **Semântica de Cópia:** Garantir que os dados em disco correspondam ao valor exato que a aplicação enviou no momento da chamada, mesmo que ela mude as variáveis logo em seguida.

Prof. Filipo Novo Mór • Sistemas Operacionais

24 Spooling e Reserva de Dispositivo

Gerenciamento de Acesso Exclusivo:

- **Spool (Simultaneous Peripheral Operations On-Line):** Intercepta e armazena saídas em arquivos temporários em disco quando o periférico de destino não aceita fluxos intercalados.
- **Reserva de Dispositivo:** Garante alocação exclusiva via software. Exige cuidado extremo no projeto do Kernel para evitar a ocorrência de **Deadlocks** (Impasses).

Exemplo Prático: O Spooler de Impressão evita que páginas de dois documentos diferentes saiam misturadas fisicamente na mesma impressora.

Prof. Filipo Novo Mór • Sistemas Operacionais

25 Taxas de Transferência de Dispositivos

Disparidade de Desempenho:

- Os hardwares operam em escalas radicalmente diferentes.
- Teclados e mouses operam na faixa de meros bytes/kilobytes por segundo.
- Barramentos de memória RAM de alto desempenho processam dezenas de gigabytes por segundo.
- Essa disparidade de milhões de vezes justifica todas as técnicas de caching e buffering do Kernel.

Prof. Filipo Novo Mór • Sistemas Operacionais

26 Tratamento de Erros e Proteção

Resiliência e Segurança do Sistema:

- **Tratamento de Erros:** O SO gerencia falhas transitórias e expõe o código de falha nas aplicações via variável de sistema padrão (`errno`).
- **Proteção de E/S:** Todos os comandos e instruções físicas de E/S mapeadas em hardware são categorizadas como **instruções privilegiadas**. Processos comuns de usuário obrigatoriamente precisam pedir permissão e passar pelas chamadas do Kernel.

Prof. Filipo Novo Mór • Sistemas Operacionais

27 Estruturas de Dados do Kernel

Mapeamento Interno de Estado:

- O Kernel precisa manter em tempo real tabelas de controle estruturadas.
- Contém registros ativos de: tabelas globais de arquivos abertos, descritores de sockets de rede em execução e estados de drivers de caractere pendentes.

Prof. Filipo Novo Mór • Sistemas Operacionais

28 Ciclo de Vida de uma Requisição

A Jornada Completa do Dado:

- Uma chamada de leitura unifica todas as camadas anteriores: o processo faz a requisição, o Kernel avalia o cache e buffers, o driver assume a tarefa de controle, o DMA preenche os blocos em segundo plano, uma interrupção avisa a CPU e o processo original é acordado com os dados limpos em seu espaço de memória.

Prof. Filipo Novo Mór • Sistemas Operacionais

29 O Mecanismo STREAMS

Modelo Unix Flexível e Modular:

- Estabelece uma conexão *full-duplex* (bidirecional simultânea) contínua entre um driver físico e o processo de usuário.
- Consiste em um arranjo de três componentes empilhados: **Stream Head**, **Módulos STREAMS** intermediários intercambiáveis e o **Driver End** conectado ao dispositivo.
- Opera de forma elegante por passagem de mensagens entre filas internas de leitura e escrita.

Prof. Filipo Novo Mór • Sistemas Operacionais

30 Fatores de Desempenho

Regras de Otimização de Sistemas de E/S:

- Reduzir drasticamente o número de trocas de contexto (*context switches*) e cópias de dados desnecessárias na memória RAM.
- Gerenciar e dosar o uso de interrupções de hardware com inteligência (adotando polling temporário quando o dispositivo for rápido o bastante).
- Explorar e maximizar o uso do controlador de DMA para poupar ciclos de processamento da CPU central.

Prof. Filipo Novo Mór • Sistemas Operacionais

31 Conclusão e Dúvidas

Encerramento do Capítulo 13:

- **Referência Principal:** SILBERSCHATZ, Abraham. Operating System Concepts. 9ª Edição.
- Espaço aberto para debate acadêmico, consolidação dos conceitos apresentados e resolução de dúvidas de estudantes.

Bons estudos! Revisem as diferenças de fluxos bloqueantes, não bloqueantes e assíncronos para a avaliação.

Prof. Filipo Novo Mór • Sistemas Operacionais